IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received January 22, 2020, accepted February 4, 2020, date of publication March 13, 2020, date of current version March 27, 2020.

Digital Object Identifier 10.1109/ACCESS.2020.2980523

Detecting Low-Rate Replay-Based Injection Attacks on In-Vehicle Networks

SATYA KATRAGADDA[1], PAUL J. DARBY, III[2], (Member, IEEE), ANDREW ROCHE[2], AND RAJU GOTTUMUKKALA[1,2]

[1]Informatics Research Institute, University of Louisiana at Lafayette, Lafayette, LA 70506, USA
[2]College of Engineering, University of Louisiana at Lafayette, Lafayette, LA 70506, USA

Corresponding author: Raju Gottumukkala (raju@louisiana.edu)

This work was supported in part by the National Science Foundation under Grant CNS-1429526, and in part by the 2016 DOE Grid Modernization Laboratory Consortium for Diagnostic Security Modules for Electric Vehicle to Building Integration under Grant GMLC 0163.

ABSTRACT The lack of security in today's in-vehicle network make connected vehicles vulnerable to many types of cyber-attacks. Replay-based injection attacks are one of the easiest type of denial-of-service attacks where the attacker floods the in-vehicle network with malicious traffic with intent to alter the vehicle's normal behavior. The attacker may exploit this vulnerability to launch targeted low-rate injection attacks which are difficult to detect because the network traffic during attacks looks like regular network traffic. In this paper, we propose a sequence mining approach to detect low-rate injection attacks in Control Area Network (CAN). We discuss four different types of replay attacks that can be used by the adversary, and evaluate the effectiveness of proposed method for varying attack characteristics and computational performance for each of the attacks. We observe that the proposed sequence-based anomaly detection achieves over 99% f-score, and outperforms existing dictionary based and multi-variate Markov chain based approach. Given that the proposed technique only uses CAN identifiers, the techniques could be adaptable to any type of vehicle manufacturer.

INDEX TERMS Vehicular networks, CAN bus, intrusion detection, injection attacks, sub-sequence mining.

I. INTRODUCTION

Vehicles are now increasingly automated and connected to Internet, so their cyber-security risks are correspondingly increasing. The vehicle's CAN bus originated during a time when no external network connections were available. The CAN bus was not designed to incorporate modern authentication and encryption solutions that are common to externally connected systems having mature cyber-security provisions. These issues in addition to the ability to connect to external networks or devices expose the vehicle's electro-mechanical systems to cyber-security threats. These cyber-physical threats are well-documented in literature [1]–[3]. Several researchers have shown attackers can potentially exploit this interface to take control of a connected vehicle, launch Denial of Service (DOS), or launch spoofing attacks on a vehicle [4]. Recent hacks on Tesla vehicles and other real-world attack demonstrations like disabling vehicular safety critical features highlights the importance of this problem [5]. Researchers and automobile manufacturers are investigating ways to enhance the cyber-security of connected vehicles through improving the CAN BUS and ECU designs, adding encryption and message authentication to provide security. However, the CAN bus infrastructure of older vehicles would still remain vulnerable to these types of attacks.

Both statistical and machine learning techniques have been applied for detecting injection attacks by observing CAN bus identifiers information. Typical information used by these techniques include inter-signal arrival time, message frequency or message sequences [6]–[8]. These techniques assume that attackers would launch a Denial-of-Service attacks (i.e. the attacker floods the bus with messages intended to override legitimate messages in the CAN bus [9], [10]. Machine learning methods have been applied on injection attacks, but these techniques have not been tested against low-rate attacks [11]. Detecting low-rate attacks is non-trivial in the domain of network security [12], [13].

The associate editor coordinating the review of this manuscript and approving it for publication was Xiaofan He.

The same is true for CAN networks. A sophisticated attacker, with knowledge about the vehicle and its CAN messages can inject very small numbers of targeted messages to achieve a desired outcome, e.g. the attacker can override the anti-theft system by sending an ''off'' command to the CAN once an ''on'' command is noticed in the CAN bus message. These attacks are illustrated by Hoppe *et al.* [14], [15]. Another example of targeted replay is demonstrated on Cherokee Jeep, where the vehicle is forced to stop on an interstate [16]. Miller *et al.* show that the Jeep can be forced to accelerate and decelerate by simulating the cruise button press on the steering of the vehicle [17]. Koscher *et al.* provide various examples of attacks that can be performed by replaying specific messages on the CAN bus [18]. Petit and Shladover investigate various attack surfaces on the vehicle that cannot be detected by a human or anomaly detection methods [19]. The sophisticated knowledge of attacker and the lower rate of these attacks which cannot be detected by traditional intrusion detection techniques makes attack detection quite cumbersomee.

We propose the use of frequency-pattern based subsequence mining approach to detect low-rate attack injections on CAN bus. This approach is based on sequence mining that has been successfully applied to detect anomalous behaviour in various domains like DNA sequences, transaction behaviour, and click sequences [20]–[22]. Since each ECU on the CAN bus generates messages based on its own internal clock at regular intervals, this behaviour results in a small set of repetitive patterns in the CAN bus. We adapt the sequence mining approach to extract these temporal sequences from CAN bus messages. Subsequence mining allows us to identify various subsequences and the rate at which they appear during the normal operation of a vehicle. The proposed approach is compared against various anomaly detection techniques using the data collected from CAN networks in an electric vehicle. The key contributions of our paper are the following:

1) Discuss different types of replay injection attacks: dominant id replay, random id replay, sequence replay, and targeted replay injections.
2) Application of frequency-pattern based subsequence mining algorithm for detecting sophisticated injection attacks (i.e. replay attack injections) in CAN bus sequences.
3) Analyze the proposed approach for various types of attacks focusing on low-rate injection attacks.

The remainder of this paper is structured as follows: Section 5 presents a brief overview of existing anomaly detection work in the area of cyber-security and CAN bus. Section 3 contains background on replay attacks, the attack generation model, and subsequence mining. Section 4 describes the proposed subsequence mining approach to detect anomalies in CAN bus data. Section 5 describes the experimental setup, various attack scenarios, the data, and evaluation criteria. Finally, Section 6 discusses the conclusions drawn from experiments, and potential future work.

II. RELATED WORK

Given the vulnerability of CAN bus with respect to lack of encryption and authentication, many researchers have proposed different anomaly detection techniques for detecting injection attacks. A comprehensive survey of anomaly detection can be found in [1], [23], [24]. This includes rule/frequency-based techniques, sequence-based and machine learning-based techniques.

Rule-based approaches use thresholds or frequency statistics to detect injections. Moore *et al.* designed an approach that exploits the regularity of the CAN bus messages, and frequency change [6]. In this case, frequency is computed as a factor of time difference between consecutive messages. There have been various other techniques designed that uses the CAN bus messages as well [1], [4], [25]. These techniques use the message ids from CAN bus sequences and the time associated with the messages to compute frequencies. The main strength of rule-based approach is low computational overhead. However, these messages are not very reliable with respect to detecting small scale targeted intrusions because these techniques do not account for arbitration in the CAN bus. Young *et al.* observed that the frequency of CAN bus messages change significantly due to changes in time intervals during the transitions of vehicle driving models and thus simple frequency or rule based models cannot be effective at detecting intrusions.

Sequence-based approaches maintain a dictionary [8] or transition matrices [26] to identify invalid sequences. In this case, an invalid transition is considered an anomaly. Dictionary-based approaches have low computational overhead of O(n), but these methods perform poorly for replay injections where a valid sequence is replayed. The transition matrix approach uses multi-variate Markov chain process that has better intrusion detection performance compared to dictionary-based approaches especially when the size of injection is long. Transition-based intrusion detection methods has high computational complexity ($O(nm^2s^2)$). Sequence based approaches on the other hand mainly use message ID sequences, hence these techniques can be potentially generalized to any type of vehicle. These techniques rely on the sequence of message ids to build knowledge models and do not require any additional information. More recent approaches used state space based approaches and hidden Markov models to determine intrusion attacks on the vehicle [27], [28]. However, identifying states on a vehicle is a cumbersome process that requires a domain expert to build the models. Moreover, these techniques are quite expensive to train and implement in a near-real time intrusion detection system.

Various machine learning based approaches have been proposed to detect injection attacks on CAN bus. Taylor *et al.* used one class SVM classifier that relies on the history of message IDs [7]. More sophisticated deep learning based

LSTM models were also applied to detect replay attacks on the CAN bus data [29]. Given that vehicle's More recent work uses a combination of a rule based and a artificial neural networks to reduce the computational cost of detecting attacks [30]. These machine learning and hybrid techniques require a lot of computation power and training data to build reasonably accurate models. There have been some solutions to develop hybrid models that extract features in real time and use cloud resources to identify anomalies [31], [32]. However, cloud based techniques require reliable communication with the cloud. The performance of all machine learning techniques is highly dependent on the training data that is used to generate the machine learning model. While modest of these techniques show good detection performance on simulated data, any attack that is not in the training model goes undetected.

Existing approaches to detect intrusions in CAN bus data fail to account for low-rate injections, are computationally expensive, or require rich extensive labelled data to detect anomalies. In order to detect highly targeted intrusions in real-time, we propose a subsequence mining approach that uses patterns of subsequences and their frequencies during the normal operation of the vehicle. This technique exploits the highly timed nature of the CAN bus messages to identify message patterns that are observed during the operation of vehicle during various stages. In addition, this technique does not require any attack data, and relies on the normal CAN bus data to identify intrusions.

III. BACKGROUND

In this section, we describe the attack model, various types of attacks, and the basic definitions of subsequence mining. The attack model describes the resources and the knowledge required by a attacker to launch an attack on a vehicle. The types of attacks describe various types of attacks launched by the attacker. Finally, we introduce important definitions from sequence mining that can be used to detect anomalous behaviour from discrete sequences.

A. ATTACK MODEL

The attacker is assumed to have knowledge of individual messages and message data fields in a CAN bus on a vehicle. In our model, the attacker has physical through the On-board Diagnostics (OBD) port, or remote access to the CAN bus through an On-Board Unit (OBU), which allows the attacker to both listen and send packets to the CAN bus. This allow the attacker to store and identify individual messages, inject a single message, or inject a sequence of messages to induce malfunctions or perform actions the vehicle would not normally perform in its current state. In light of these assumptions, we consider four different replay attack scenarios termed the: dominant id replay, random id replay, sequence replay, and targeted replay injections.

1) DOMINANT ID REPLAY

An attacker can inject high priority messages on to the CAN bus. The attacker broadcasts the lowest message id on the CAN bus in a relatively short higher frequency cycles. Since the lowest ID is used, these messages are prioritized during the arbitration process compared to other messages with higher message IDs. Because all ECUs share the bus, losing the arbitration increases the latency for legitimate messages. Moreover, the complete loss of some CAN bus messages can occur. These effects may cause the vehicle to become unresponsive to the driver's actions or commands. This attack also results in a denial of service. A targeted attack of this kind, using the right frequency can paralyze critical vehicle functions [33]. An example is shown in Figure 1. In this example, the dominant id 2 is repeatedly inserted into the CAN bus, and the other ECUs wait for the dominant id message to be transmitted before their messages are transmitted in the CAN bus. Because the attacker only needs to know the lowest ID in a CAN bus, this attack type is considered a naïve attack.

2) RANDOM ID REPLAY

In this scenario, the attacker injects messages previously seen, but in random order, significantly increasing the processing time for each ECU and the bandwidth of the CAN bus. Moreover, the random messages confuse the vehicle's ECUs, resulting in unpredictable and unintended consequences. Various scenarios involving these attack types are presented in [18]. This attack is also rather simple, because the attacker only needs to sniff CAN traffic to get a valid list of message IDs, and then randomly replay it. Figure 1b shows an illustration of Random ID Replay. In this example, the attacker inserts messages with known message ids (2, 130, 174, 292, and 421) at random intervals.

3) SEQUENCE REPLAY INJECTION

With the Sequence Replay Injection, the attacker injects onto the bus, not a single ID but a sequence of observed message IDs with various frequencies, with the goal of recreating a series of driver actions, e.g. braking, steering turns, acceleration changes, or disabling the airbag. These messages to ECUs cause unintended and potentially hazardous vehicle behaviors. An example is presented in [15]. To launch this type of attack, the attacker needs to capture the CAN's original messages and understand their potential impact on the vehicle, to enable injection at the right moment to severely impact and potentially damage the vehicle. An example is presented in Figure 1c. The attacker inserts a known sequence of messages (174, 292, 2) at random messages. These types of messages can easily fool a dictionary based approach or a sequence based approach like Markov chain process.

4) TARGETED REPLAY INJECTION

In this type of attack, the attacker injects the CAN with specific messages at specific times to trick the vehicle. An example is presented in Figure 1d. Assume message 2 reports the current vehicular speed and 421 reports the vehicle's state (e.g. parked, neutral, drive, or reverse). In this sophisticated attack, the attacker reads the CAN messages when the

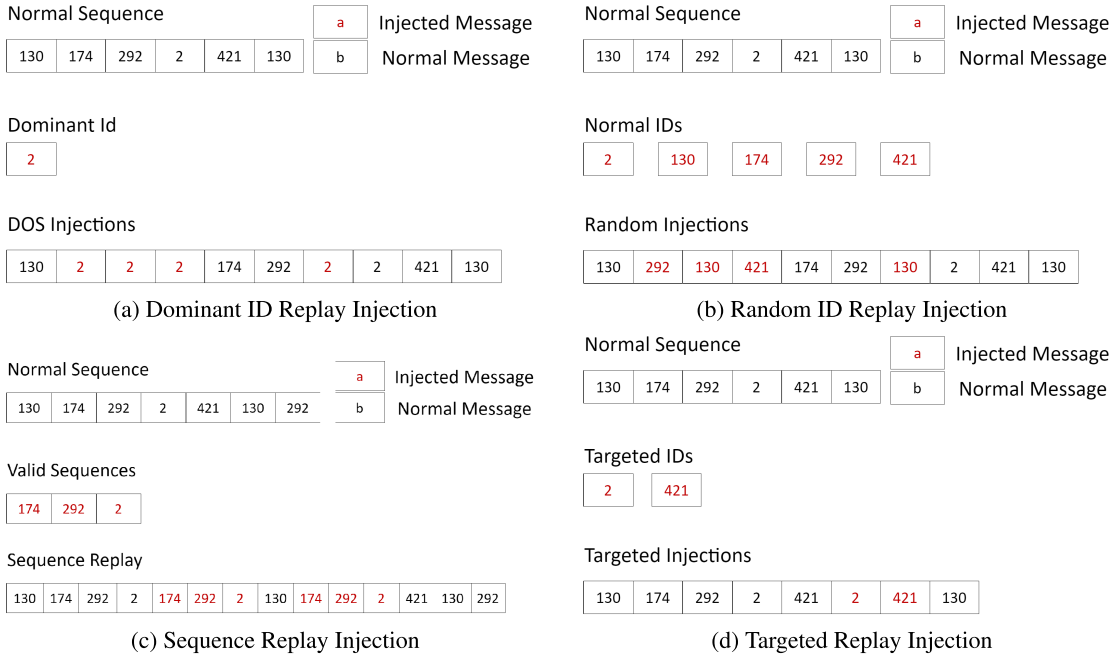


FIGURE 1. Illustration of various injections attacks.

original 2 and 421 are transmitted (e.g speed is 50 mph and state is drive). Then the attacker replays these same messages, but with modified data fields corresponding to a speed of 2 mph, and putting the vehicle in park, triggering the car to stop suddenly while on the highway at 50 miles per hour. Another targeted replay attack would turn off the car's lights. Here, the attacker inserts a message to turn off the lights as soon as the normal message to turn the lights on is transmitted, forcing the vehicle to keep the lights off. An overview of targeted attacks is presented in [15]. Here, the attacker has sufficient knowledge of valid message sequence combinations, to allow him to insert apparently correct sequences onto the bus. Targeted replay attack is a low-rate attack that cannot easily be detected by existing intrusion detection techniques.

B. SEQUENCE MINING

Sequence mining pertains to the discovering of interesting subsequences among a set of sequences, where the interestingness of a subsequence is measured in terms of its occurrence frequency and length. We introduce sequence mining concepts from Fournier-Viger *et al.* [34]:

Definition 1 (Sequence): Let $I = \{i_1, i_2, \ldots, i_l\}$ be a set of all items. An itemset $Ix = \{i_1, i_2, \ldots, i_m\} \subseteq I$ is a nonempty and unordered set of distinct items. A sequence s is an ordered list of itemsets or events denoted as $< I_1, I_2, I_3, \ldots, I_n >$ such that $I_k \subseteq I (1 \leq k \leq n)$.

Definition 2 (Sequential Database SDB): A sequential database SDB is a set of sequences, i.e., $SDB = \{s_1, s_2, s_3, \ldots, s_p\}$, where s_j is a sequence, $1 \leq j \leq p$.

Definition 3 (Sequential Containment): A subsequence $S_a =< A_1, A_2, \ldots, A_n >$ is said to be contained in a subsequence $S_b =< B_1, B_2, \ldots, B_m >$ if and only if there exist integers $1 \leq i_1 \leq i_2 \leq \ldots \leq i_n \leq m$, such that, $A1 \subseteq B_{i1}, A2 \subseteq B_{i2}, \ldots, An \subseteq B_{in}$ and this is denoted as $S_a \subseteq S_b$. In this case S_a is considered as a sub-pattern of S_b and S_b is also said to be super-pattern of S_a.

Definition 4 (Support): The support of a subsequence S_a in a sequential database *SDB* is determined by the number of sequences $S \in SDB$, such that, $Sa \subseteq S$ and it is denoted by $supSDB(S_a)$.

$$supSDB(S_a) = \frac{f(S_a)}{l_s} \tag{1}$$

where $f(S_a)$ is the frequency of the subsequence S_a and l_s is the total number of sequences in the current time period. In sequence mining, the complete set of maximal subsequences (super-patterns) among all the sequences need to be extracted from the sequential database *SDB*.

IV. METHODOLOGY

We implement an subsequence mining based anomaly detection algorithm that uses CAN message frequency patterns to detect intrusions. The CAN bus messages are gathered using a on-board unit (OBU) that listens to the CAN bus and collects these messages in real-time. These messages are then processed as a continuous data stream of tumbling windows divided based on a specific time period, where each window contains the set of messages collected during a single time period. The subsequence mining algorithm extracts subsequences from the CAN bus messages in each window. A training phase is used to build a dictionary to model the normal behaviour of the vehicle. This dictionary is deployed

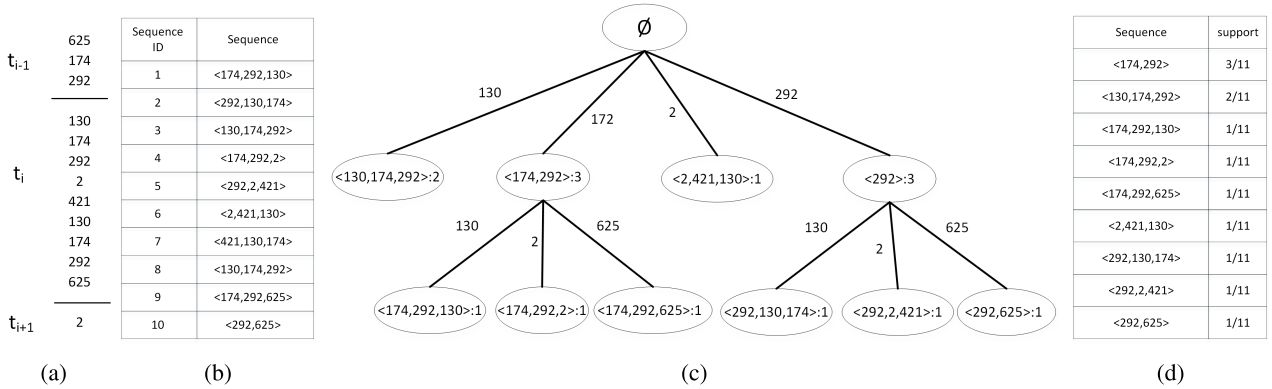


FIGURE 2. **Various steps in extracting sequences and their support scores for a single time period of data. (a) Original sequence from CAN bus, (b) Sequence dictionary generated from base sequence, (c) Frequent sequence tree populated from the sequence dictionary, (d) Subsequence rules and support.**

during the detection phase to detect intrusions in the CAN bus. We describe the subsequence mining algorithm, training phase and the detection phase in detail below.

A. SUBSEQUENCE MINING ALGORITHM

The main goal of the subsequence mining algorithm is to process a discrete sequence of CAN bus messages and extract all the subsequences in that sequence along with their support values. The subsequence mining algorithm is a four step process: Sequence extraction, populating the sequential database, creating a frequent sequence tree, and subsequence rule generation.

1) SEQUENCE EXTRACTION

In the sequence extraction process, a temporal sequence of messages are created from CAN bus messages. All the messages that are generated by the ECUs are transmitted through the CAN bus. The OBU listening to the messages collects these messages in real-time. All the messages collected during a single time period t_i is considered to be the base sequence of messages. All the messages are ordered based on the time these messages were collected by the OBU. On order to maintain the continuity of messages, the last $n-1$ messages from the previous time period are added to the base sequence, where n is the maximum length of the subsequence. If l_i is the total number of messages active in the CAN bus during a single time period t_i, the total number of messages in the base sequence b_i is $l_i + n - 1$. An illustration of the sequence generation is presented in Figure 2a. All the messages in the time t_i are considered to be the sequence of messages, the two messages from the two message ids (n=3), 174 and 292 from the previous time period are appended to the current sequence to form the base sequence

2) SEQUENTIAL DATABASE GENERATION

The sequential database contains a collection of all sequences extracted from the base sequence as defined in Definition 2. In the sequential database generation, a set of maximum subsequences are generated from the base sequence. A sliding window of size n is used to extract all the subsequences of size n. In order to account for the last $n-1$ messages from the base sequence, all subsequences of length greater than 1 are also added to the sequence database. For a sequence generated from a set of messages of length l, the total number of maximal sequences generated will be $l+n-2$. An example of sequential dictionary generated from the base sequence in Figure 2a is illustrated in 2b.

3) BUILDING THE FREQUENT SEQUENCE TREE

In order to achieve sequential containment, a frequent sequence tree is generated. Each node in the sequence tree contains a subsequence and the number of maximum subsequences this subsequence appears in. Each parent node is a sub pattern of its child, where the child node is more specific than its parent, with the subsequence in the parent node forming the prefix of the subsequence in child node. For example, all the children of $< 174, 292 >$ has that particular subsequence as its prefix. All the edges of the sequence tree are ordered based on suffix of the child nodes. This enables fast lookup of the rules. All the nodes in the sequence tree are considered valid subsequences from the CAN bus sequence extracted from a single time period. Figure 2c presents the sequence tree generated from all the sequences in the sequence dictionary. Some of the sequences are sub-patterns of the other sequences. For example, $< 292 >$ with frequency 3 is a sub-pattern of $< 174, 292 >$ with the same frequency. The details of the process used to generate the sequence tree is presented by Wang and Han [35].

4) EXTRACTING SUBSEQUENCE RULES

All the subsequences represented as a node in the sequence pattern tree are extracted into a subsequence list by parsing the sequence pattern tree. All the sub-patterns that are marked for sequential containment are removed from the subsequence list. A support score is calculated based on the formula presented in Equation 1. All the subsequences and their support scores are saved in a subsequence dictionary.

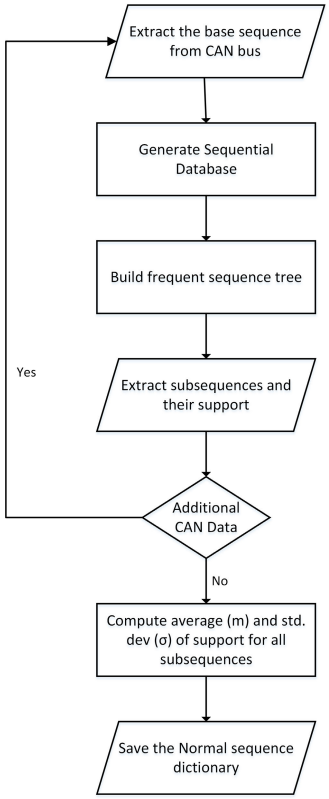


FIGURE 3. Flowchart of the training phase to generate a dictionary of all valid subsequences, mean and standard deviation of their support scores.

B. TRAINING PHASE

The main goal of the training phase is to model the normal behaviour of the vehicle, which can then be used to detect intrusions in the CAN bus. The training phase is an offline process, where the data is gathered from a vehicle and subsequence rules are generated from each time period of data. All the unique subsequences are extracted from the support dictionary from each time period. The average (m) and standard deviation (σ) of support values for each of the unique subsequences are computed. The distribution of $m \pm 4\sigma$ represents the range of support score for a subsequence during the normal operation of the vehicle with a confidence of 99.994. The final subsequences, average and standard deviation of the support values are stored in the normal subsequence dictionary. The flowchart for the complete training phase is provided in Figure 3.

C. DETECTION PHASE

In the detection phase, a set of consecutive message ids from a single time period are extracted and evaluated for intrusions in real-time. Similar to the training phase, a sequence database is generated based on the maximum subsequence length from a single time period of CAN bus data. The frequent sequence tree is generated to compute all the subsequences adhering to sequential containment. The resulting set of subsequences and their support scores are stored in the support dictionary. Each subsequence rule s_x in the support dictionary is compared to the subsequences in the normal subsequence dictionary. If the s_x is not present in the normal subsequence dictionary, the rule is labelled anomalous. If s_x is present in the normal subsequence dictionary, it is considered normal if $m - 4\sigma \leq s_{sx} \leq m + 4\sigma$, where s_{sx} is the support of a subsequence s_x, m is the average support for that subsequence and σ is the standard deviation of the subsequence in the normal subsequence dictionary. The flowchart for the complete training phase is provided in Figure 4.

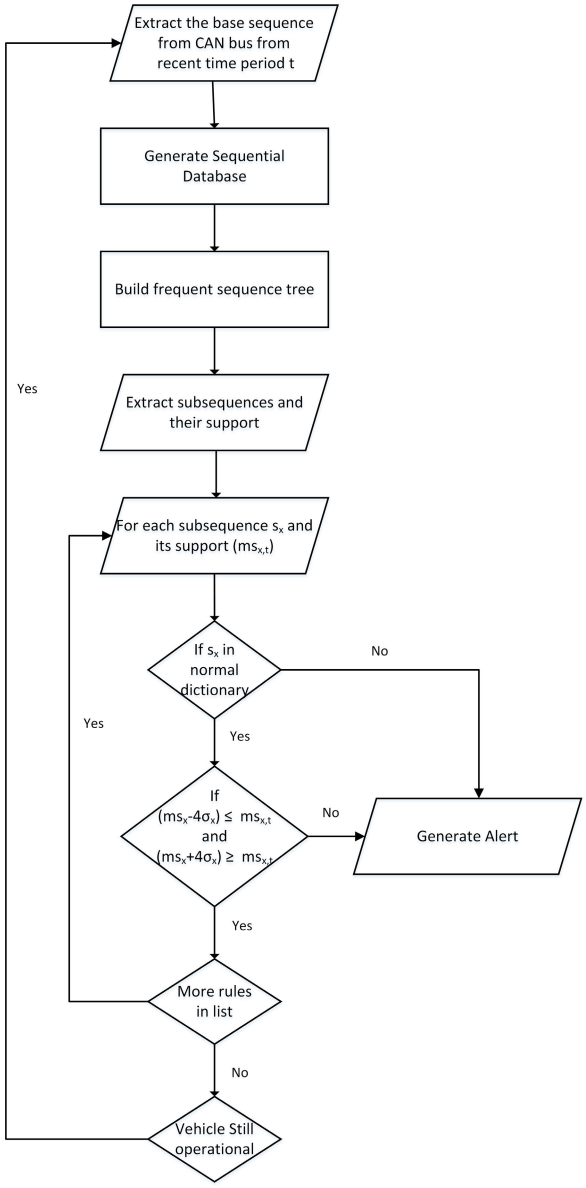


FIGURE 4. Flowchart of the testing phase to detect intrusion and generate alert based on the normal dictionary in training model.

V. EXPERIMENTAL SETUP

A. DEPLOYMENT

This intrusion detection model is deployed on a On Board Unit (OBU) that hosts the subsequence mining algorithm. The OBU is a Raspberry Pi III model A that is connected

to the CAN bus through the OBD-II port of the vehicle. The data from the CAN bus is accessed in real-time using the OBD-II Port of the vehicle and a OBD Breakbox to access the CAN pins for the primary and EV CAN buses. The data is gathered from the CAN bus is translated into machine readable format using PyCAN python library. The intrusion detection algorithm is also designed using python 3.5. The intrusion detection algorithm deployed on the OBU makes a determination at every time period if the previous time period of data contains any intrusions.

B. DATA COLLECTION

In order to evaluate the proposed approach, we collected the data from a 2012 Nissan leaf. We gathered the data from both the primary and EV CAN buses of the vehicle using the OBD-II Port of the vehicle and a OBD Breakbox to access the CAN Pins for the primary and EV CAN buses. We connected the OBD-II port to our lab computer running the python 3.5 with PyCAN library. We were able to both read the data from the CAN bus and also inject the data into the CAN bus using the PyCAN library. We gathered the data from the vehicle over multiple scenarios that include when the car is driving, charging, and stationary. In total we gathered a total of 1 hour 13 minutes of CAN data from the primary CAN and 1 hour 6 minutes of data from the EV CAN.

The data is then divided into training and testing data where 80% of the data is considered training data and rest of the 20% data is used to test the data. The training data is used to build statistical and knowledge models for each of the baseline and proposed sequence mining approaches. The injections are inserted into the test data to generate intrusions. These messages and sequences are used to generate various attack sequences based on the attack models described in Section III. All the intrusion detection techniques are evaluated on the test data to identify the intrusions.

1) ATTACK DATA GENERATION

The attacks on the CAN data are generated based on earlier work [8], [30]. The attacks on the CAN bus requires three parameters, the injection length (l), attack frequency (a), and the insertion time (t). The injection length denotes the total number of messages in the attack sequences that will be inserted into the CAN bus. The attack frequency is the total number of injection attacks of length l inserted into the CAN bus in a given time period. Insertion time $t = \{t_1, t_2, \ldots, t_a\}$ is the relative time when the messages in a single attack are inserted into the CAN bus from the start time of the CAN bus message. New injections are inserted into the existing CAN bus messages starting at t_i, $1 \leq i \leq a$, every time the CAN bus is free.

Three different types of attack sequences are generated based on the attack model described in Section III. To generate a dominant id replay sequence, we inject a message with the most dominant message id from the CAN bus. The attack sequence is created with length l, where the most dominant id appears a total of l times. For a random injection sequence, a set of l prefetched message ids are randomly selected from the training data and injected into the CAN bus during the insertion time. Finally, in the subsequence attack, a contiguous sequence of messages of length l are selected from the training data. These sequence of messages are inserted into the test data. All the messages in an attack sequence are inserted immediately after the idle time on the CAN bus so that the messages are inserted in order.

We also perform a targeted replay injection to inject messages after the messages with the same message id are transmitted. The length of the attacks are 1, 2, and 3. The frequency of the messages are the frequency of the original messages in the CAN bus. The details of the targeted replay are presented in Section III-A4.

C. BASELINE

We compare the performance of the proposed sequence mining approach to detect anomalous CAN bus sequence to three approaches in literature: a frequency based approach, a dictionary based approach and a multivariate higher order Markov chain process.

1) FREQUENCY BASED APPROACH

The total number of messages are calculated at the end of every time period during the training phase. At the end of the training phase, we collect information the mean (m) and standard deviation (σ) of the total number of messages at every time period in the training phase. This number is considered to be the usual number of messages during the normal operation of the vehicle. In the testing phase, if the total number of messages at a time period appears outside a threshold ($m \pm 4\sigma$), then it is considered an intrusion. This work is based of work in [6].

2) DICTIONARY BASED APPROACH

In the training phase, a transition matrix of order n is built, where n is the total number of unique IDs on CAN bus data. Each sequence of IDs in the training data are analyzed and are marked as valid transitions in the transition matrix. This transition matrix contains a dictionary of all legitimate sequences in the training data. In the testing phase, each sequence is compared to the dictionary. If the transition matrix marks the sequence as true, it is considered a legitimate sequence and is marked as an anomaly if it is false. A set of messages is considered to contain an injection if any subset of the messages is labelled an anomaly. This technique is introduced in [8].

3) HIGHER ORDER MULTIVARIATE MARKOV CHAIN PROCESS

Each message ID is processed as a collection of binary sequences. There are two possible states at a given time t, 0 or 1. The state of an ID is 1 if the message is active at that time and 0 if some other ID is active during that time period. In the training phase, sequences for all the IDs are generated and the transition matrices and weights for the

matrices are generated based on the work in [36]. We train a new Markov model for each ID's data sequence. In the testing phase, we predict the next state for all the IDs. The message ID is considered normal if the next state for that ID is 1 and anomalous if that next state for that ID is 0. An injection is considered to be detected if any part of the injection is labelled as anomalous. A set of messages is considered to have an injection of any of the messages is labelled as anomalous. The proposed approach is presented in [26].

All the three baseline approaches and the subsequence mining approach are evaluated to detect intrusions on CAN bus data. A sequence of messages are gathered for a time period of 1 second and tested for intrusions. Each of the techniques are evaluated using the criteria described below.

D. MODEL EVALUATION

Precision, recall and f-score are used to judge the efficacy of the replay attack detection models. Precision measures the number of true anomalies to the total number of anomalies detected by an approach. Thus, precision measures the ratio of the number of true positives to the number of true positives and false positives. A 100% precision implies that there are no false positives detected. Similarly, recall measures the number of true positives to the number of true positives and false negatives. A 100% recall means that a given approach detected all the attack sequences injected into the CAN bus. Traditionally, increase in precision leads to a decrease in value of recall and vice versa. Thus F-score is used to balance between precision and recall, where f-score is the harmonic mean between precision and recall. More details about precision, recall, and f-score is explained in the work by Elhamahmy *et al.* [37] The formula for these three approaches are presented below

$$precision = \frac{I \cap D}{D} \tag{2}$$

$$recall = \frac{I \cap D}{I} \tag{3}$$

$$F - Score = 2 \times \frac{precision \times recall}{precision + recall} \tag{4}$$

where I is the injected replay sequences in the test data and D is anomalous sequences detected by the anomaly detection algorithm. $P \times 100\%$ gives the percentage of injections detected by the model that were actually fraudulent insertions placed in the data by the injection algorithm, as opposed to false negatives, which are given by the percentage $(1 - P) \times 100\%$. $R \times 100\%$ gives the percentage of fraudulent injections inserted that were detected by the model, meaning that $(1 - R) \times 100\%$ represents the percentage of injections that avoided detection. Lastly, $F \times 100\%$ summarizes overall model performance.

VI. RESULTS & ANALYSIS

In this section, we compare the effectiveness of sequence mining approach to detect injection attacks on the CAN bus and compare it to frequency based approach, dictionary based approach, and Markov chain process. We compare the performance of these techniques to detect various types of injection attacks that include dominant id replay injection, random id replay injection, and sequence replay injections. The precision, recall, and f-score are compared for multiple injection lengths and number of injection attacks. We also compare the time taken to build a model from the training data and the time taken to detect injection attacks from a single time period of data. Finally, we also compare the effect of the maximum subsequence length for detecting injection attacks on the data.

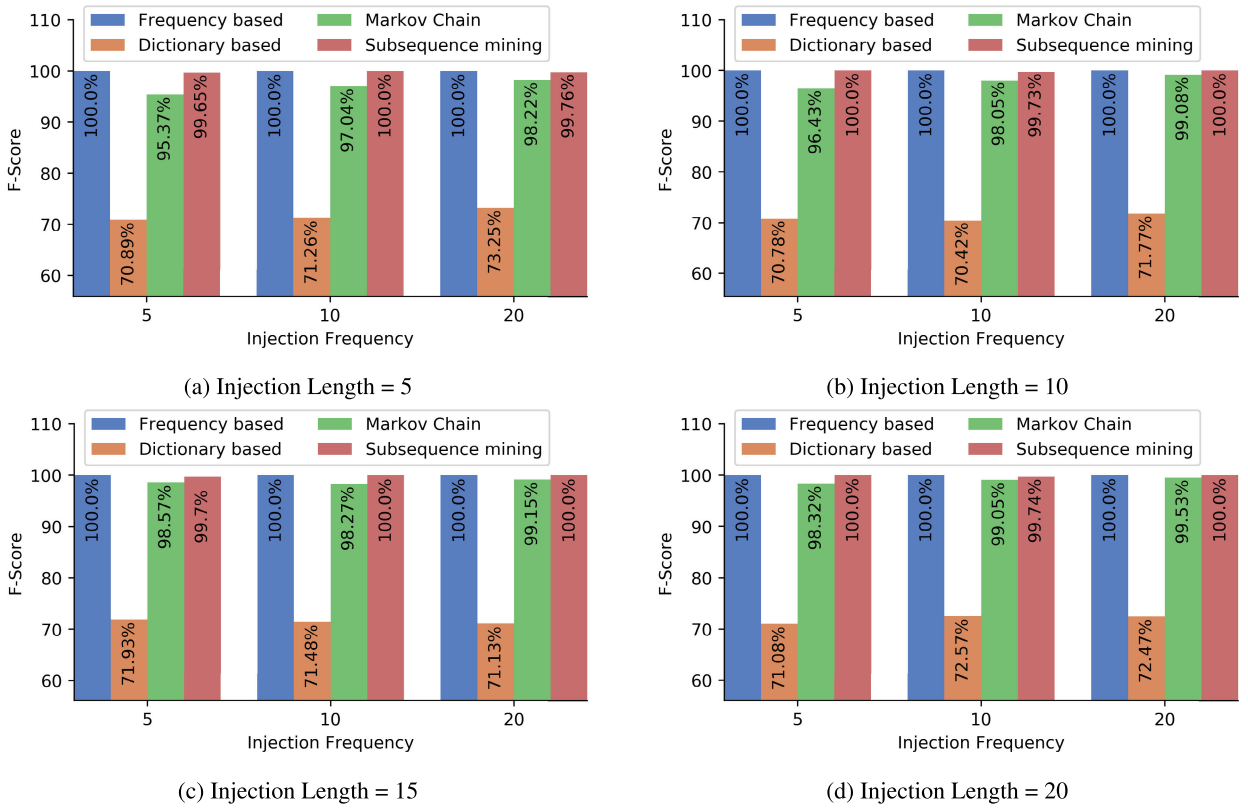
A. DOMINANT ID REPLAY INJECTION

Dominant ID replay injection is a basic denial of service attack where the attacker replays a message with the dominant id. The dominant id being replayed with a high frequency forces arbitration in the CAN bus and delays other messages. The most dominant ids in the dataset are 0×2 and $0 \times 11A$ for the primary and EV CANs respectively. We evaluated the precision, recall, and f-score to detect an injection length of 5, 10, 15, and 20 with an attack frequency of 5, 10, and 20. The F-Score for various detection techniques is presented in Figure 5.

The frequency based approach can detect all the attack injections along with 100% accuracy. The 100% accuracy can only be achieved if all the attacks were detected with no false positives or false negatives. The dictionary based approach on the other hand has no false positives, but has a 50% true negative rate. This main reason for the low recall value is that the 0×2 is a valid combination with 38 of the 48 messages on the primary CAN, therefore majority of the valid denial of service attacks are not detected by the dictionary based approach. The multivariate Markov chain process can detect most of the true positives with low number of false positives and true negatives. The results indicate that there is a slight increase in the recall values as the length of injections and the attack frequency increases. Sequence mining can detect the attacks all the injections into the CAN bus data. While both sequence mining approach and the frequency based approach can detect attack sequences in the data with 100% precision and recall values.

B. RANDOM ID REPLAY INJECTION

Figure 6 presents the different measures used to evaluate the performance of various techniques on random injection attacks. In random injection attack, random messages from training data are extracted and an attack sequence is generated from these messages. The goal of the attacker is to trick the ECUs into accepting these messages from various times and confuse the vehicle. The frequency based approach was able to detect most of the attack injections. The random injections lead to the frequency based model to label some of the low-rate injections as legitimate CAN messages. However, as the number of injections increases, the actual frequency of the ids decrease which leads to a lower false positive



(a) Injection Length = 5

(b) Injection Length = 10

(c) Injection Length = 15

(d) Injection Length = 20

FIGURE 5. F-Score of various injection detection techniques to identify dominant id injection attack for injection length of 5, 10, 15, and 20.

and true negative rates. The dictionary based attack similarly performs poorly on a smaller injection length. The number of invalid combinations increases as the frequency and length of injections increase. Similarly, Markov chain process also detects more injections with a higher injection length compared but only achieves a recall of 71.87% for injection length of 5 compared to 79.78% for injection length of 20. The number of injections does not seem to have an impact on the performance of the frequency based, dictionary based and the Markov chain process. All these three approaches are online injection attack approaches which evaluate a set of messages in a sliding window to determine if the sequence in the sliding window is valid or not. However, the sequence mining approach can detect smaller injection lengths with a recall of 98.17%, 98.54%, 99.53% and 99.3% for injection length of 5 for a attack frequency of 5, 10, 15, and 20 attacks in a time period respectively. As sequence mining approach detects anomalies on a collection of sequences extracted from a single batch of data, the performance of the sequence mining approach increases as the number of attacks also increases. As the injection length increases the precision and recall values also increases for sequence mining approach. This is due to an increase in the number of abnormal sequences and substantial changes in the support scores for the subsequences detected from injections during attacks. For a random injection, sequence mining approach outperforms all the other techniques for smaller attack sequences and outperforms dictionary based approach and Markov chain process in all scenarios.

C. SEQUENCE REPLAY INJECTION

A sequence replay attack is the scenario where a previously known sequence of messages is replayed in a wrong context to initiate an attack. This attack is a relatively subtle attack where the attacker has knowledge of the attack sequences and the attacker can inject the same message into the CAN bus during a specific time to inflict maximum damage on the vehicle. The evaluation of various techniques is presented in Figure 7.

The precision and recall using a frequency based attack is 99.17% and 99.07% on average for attack sequences of length 5. As the length of the attack sequence increases the precision and recall also increases. This improvement in detection accuracy is due to the increase in number of additional messages during the current time period in consideration. The dictionary based approach fares poorly compared to the other approaches because majority of the attack sequences contain valid sequences and the only injections that can be detected by this sequence attacks are the invalid combination of messages that appear during the beginning and the ending of the attack sequence. Markov chain process also exhibits a similar pattern where the precision and recall values remain the same irrespective of the injection length and the attack frequency. Markov chain process similar to dictionary based

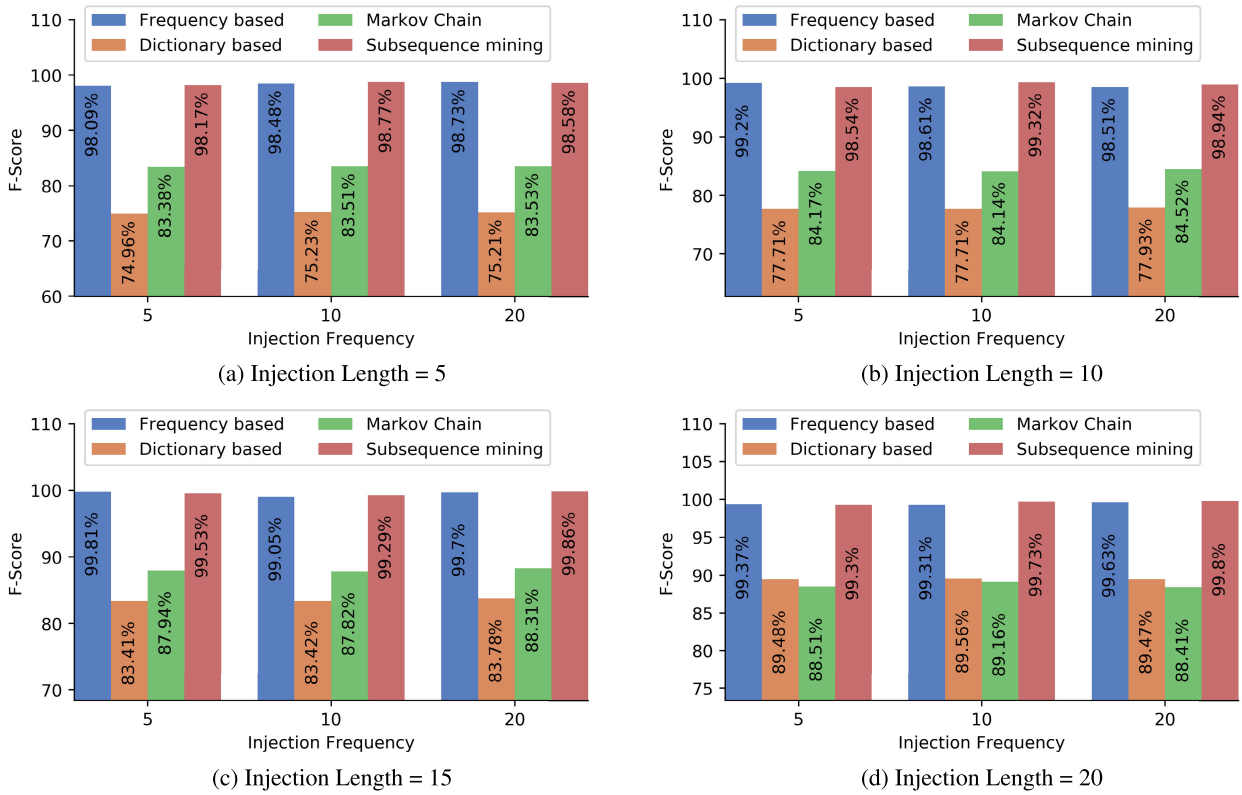


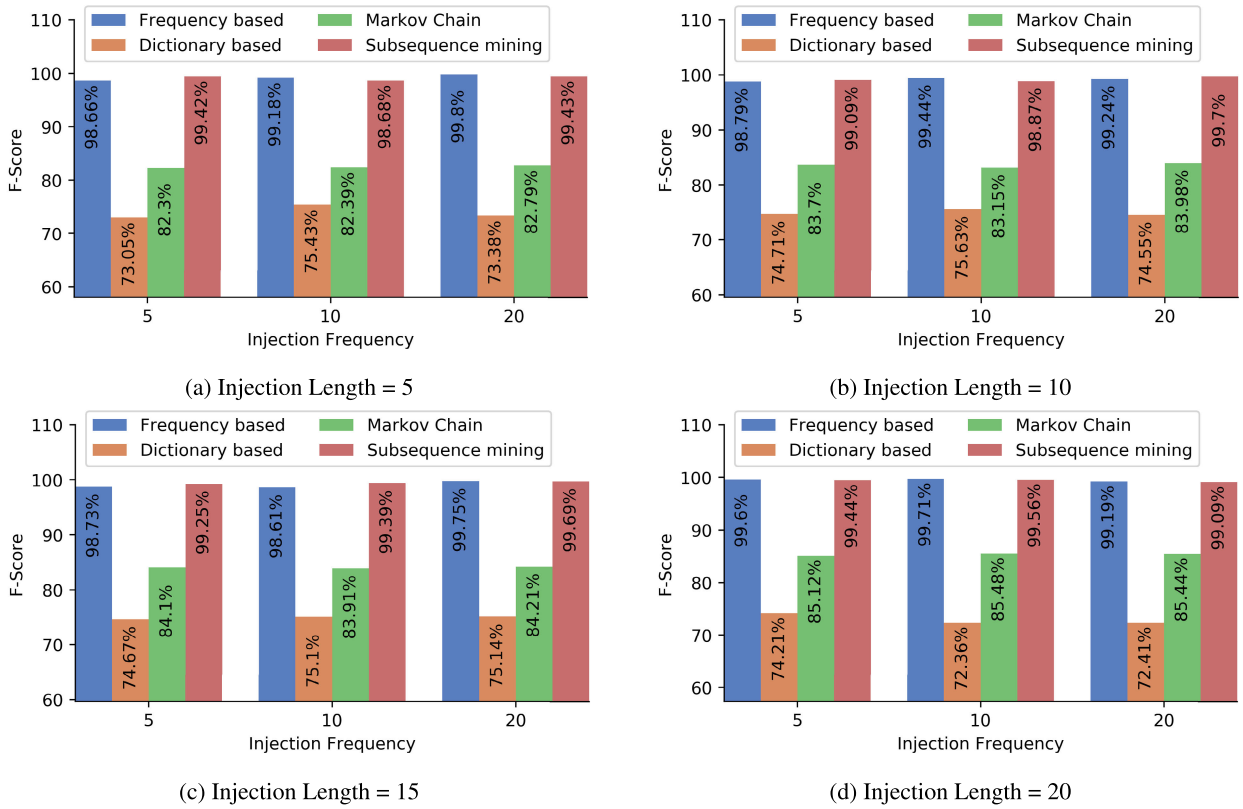
FIGURE 6. F-Score of various injection detection techniques to identify random id replay injection for injection length of 5, 10, 15, and 20.

approach can only detect injection attacks based on the start and end of the attack sequence where the consecutive ids have a lower probability of appearing together. Sequence mining approach can detect about 98.68% of the attack sequences on average for low length attacks. This is higher than frequency based, dictionary based and Markov chain where the recall values are 99.07%, 98.91%, and 98.94% respectively for an injection length of 5. As the length of the injection increases, the recall values increases to 92%. The difference between the support values from training and testing periods increase as the injection length increases. As the number of attacks increases, the number of subsequences detected and the distribution of support score changes leading to better recall values. Frequency based approach outperforms sequence mining approach for longer injections due to the reduced frequency of consecutive messages, whereas sequence mining approach outperforms dictionary based approach and Markov chain processing. The results indicate that the sequence mining approach can detect sophisticated highly precise, small injection length, and low-rate attacks with higher effectiveness compared to frequency based approach, dictionary based approach, and Markov chain process.

D. TARGETED REPLAY INJECTION

In a targeted replay injection, a set of messages are inserted into the CAN bus after the original message with the same message id is transmitted into the CAN bus. The messages are inserted with the right context, i.e. the preceding and the consecutive messages are valid transitions in the CAN bus. This is a sophisticated attack, where the attacker has knowledge about the message ids, their order, frequency, and transitions. We simulate three scenarios studied in the literature [15]. First, replay a message corresponding to the head light to turn the head light off. Second, replay the speed and state of the vehicle messages corresponding to force the vehicle into parking mode. Finally, replay messages corresponding to spoofing the speed, rpm, and radio displayed to the driver.

The F-score for frequency based, dictionary based, Markov chain process, and sequence mining are presented in Figure 8. The frequency based approach has a precision of 100% but cannot detect all the anomalous patterns in the CAN bus data. The recall of the frequency based technique is 63.57%, 71.12%, 79.73% for the three scenarios presented above. The dictionary based technique on the other hand has lower f-score due to the attacker injecting the messages in the right context. The markov chain process has a F-score of 53.02% to detect intrusion to change head lights, 55.13% to while forcing car into park, and 56.18% when posting wrong information to the driver. The subsequence mining approach on can detect targeted injections in the CAN bus with a precision of 100%. The recall for the three scenarios described are 87.15%, 92.18%, and 95.32%. The recall values for all the techniques increases as the number of injected messages increases. The subsequence mining approach can



(a) Injection Length = 5

(b) Injection Length = 10

(c) Injection Length = 15

(d) Injection Length = 20

FIGURE 7. F-Score of various injection detection techniques to identify sequence replay injection for injection length of 5, 10, 15, and 20.

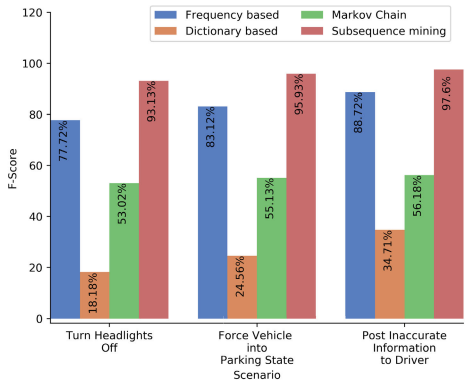


FIGURE 8. F-Score to detect targeted replay attacks using various intrusion detection techniques.

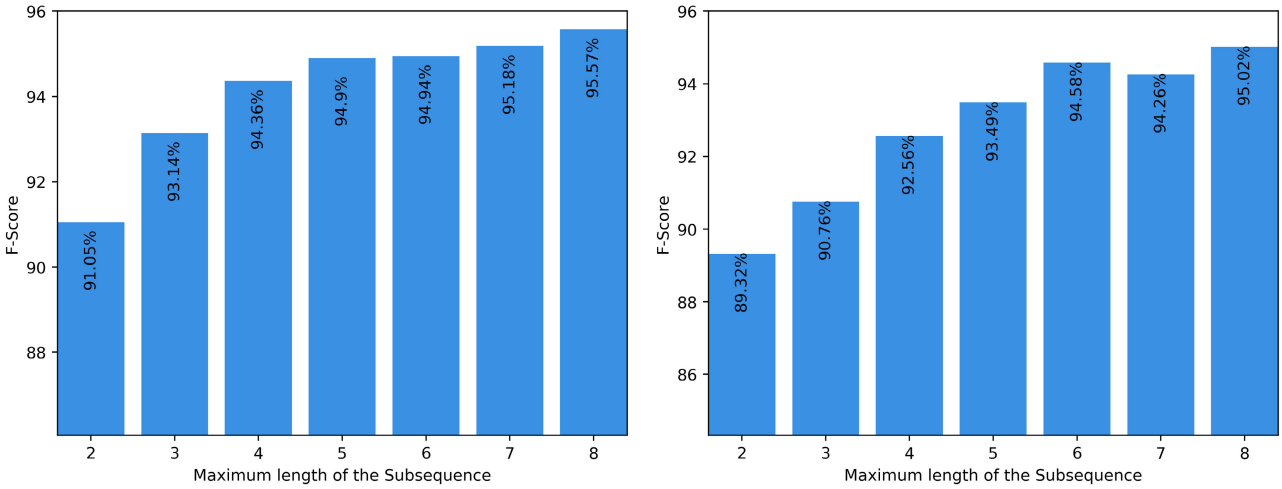
detect the messages based on both the validity of the context of messages and also the normal frequency of the detected subsequences. A valid subsequence can be considered an anomaly if the frequency of that subsequence is outside the permissible range.

E. LENGTH OF THE TIME PERIOD

We evaluated the subsequence mining approach to detect the intrusions in the data for the time window in ranges of 100 ms, 250ms, 500ms, and 1 second using the targeted replay scenario. An intrusion detection approach would be practical if it can detect an injection in a shorter time interval in order to handle the attacks immediately. The F-Score while detecting the attacks with various time periods is presented in Table 1. The precision and recall does not significantly change for the time windows of 250, 500, and 1000 milliseconds. However, the precision values are substantially lower for a window length of 100 milliseconds. This lower precision values are due to the subsequence mining not being able to capture the support scores of various subsequences. The maximum inter-message frequency in the dataset is 100 milliseconds. Some of the messages are delayed due to arbitration and latency in the CAN bus. These delays induce high variance in the support scores in the training set. The high variance distorts the support scores leading to labelling anomalous subsequences as normal. The results indicate that the minimum window length should be higher than the maximum inter-arrival time of the messages in the CAN bus.

TABLE 1. F-Score to detect intrusion in a window with different time periods using subsequence mining approach.

Window Length	Turn Head Lights Off	Force Vehicle into Parking State	Post Inaccurate Information to Driver
100 ms	85.12	86.16	87.19
250 ms	94.12	95.67	97.81
500 ms	93.75	95.13	98.04
1 sec	93.13	95.93	97.6



(a) Primary CAN

(b) EV CAN

FIGURE 9. F-Score for length of subsequence for targeted replay injection.

TABLE 2. Training and testing time for dictionary based, Markov chain, and sequence mining approaches.

Model	Training Time (sec)	Detection Time (sec)
Frequency based	4.4	0.024
Dictionary based	4.3	0.023
Markov Chain	481.4	1.3
Sequence mining	9.04	0.151

F. COMPUTATIONAL ANALYSIS

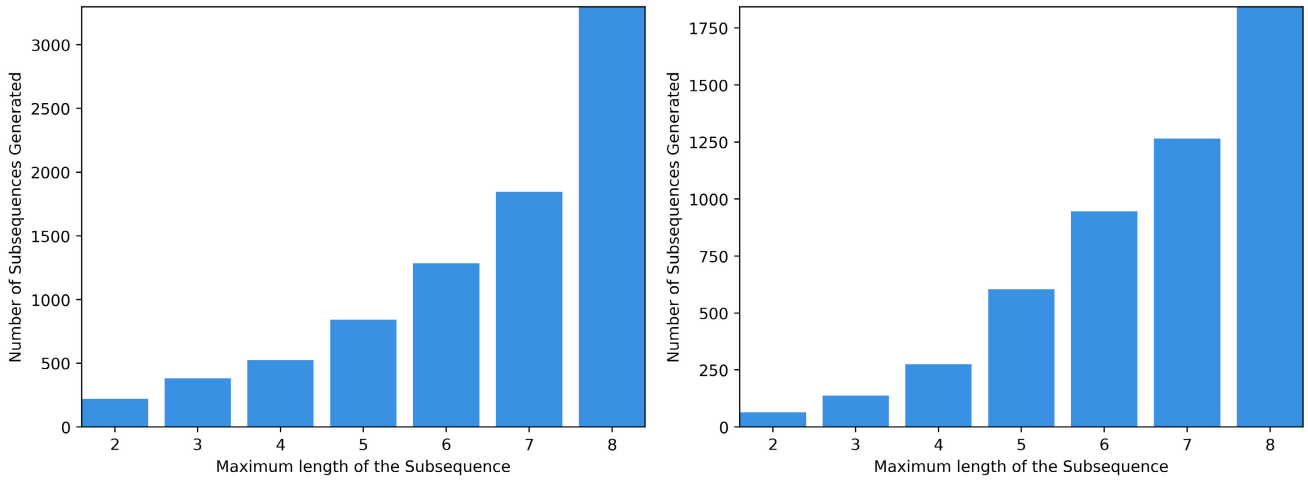
We compare the time taken to generate training models for various techniques. The time complexity of the frequency based approach is $O(n)$. The frequency based approach has a linear time complexity because the mean and the standard deviation of the messages can be computed in a single pass of data. Similarly, the dictionary based approach also has a time complexity of $O(n)$ and can generate the dictionary in real-time as the data is read from the CAN bus. The Markov chain process is computationally expensive with the time complexity reaching $O(nm^2s^2)$ where n is the length of the sequence, m is the total number of unique message ids in the sequence, and s is the total number of states per message.. The sequence mining has a total time complexity of $O(n^2)$, where the time taken to build a suffix tree for a given subsequence is $O(n)$ and the sequences needs to be iterated over to compute the support score. Table 2 presents the time taken to process the entire training data and the average time taken to detect anomalies on 1 second of test data.

The training phase is an offline process, where the models are generated for each technique. The time taken to build the model for frequency based approach and the dictionary based approach are 4.4 seconds and 4.3 seconds respectively. Both these techniques are single pass approaches that process the data in real-time. The Markov chain process on the other hand takes about 8 minutes to build the transition matrices and weights of the matrices for in the multivariate Markov chain process. Sequence mining on the other hand takes about 9 seconds to extract sequences from the training data containing 2496 base sequences, where each base sequence contains CAN messages extracted over a period of 1 second.

The detection time for one second worth of CAN bus data is about 24 milliseconds using frequency based approach and the time is about 23 milliseconds using the dictionary based approach. The Markov chain process takes about 1.3 seconds to detect intrusions for 1 second worth of CAN bus messaged. Subsequence mining on the other hand detects intrusions in about 0.15 seconds. Thus, subsequence mining can determine if the vehicle is under attack before the next batch of data is ready, therefore accomplishes near-real time performance. While both the frequency based approach and dictionary based approach can detect intrusions faster than subsequence mining, subsequence mining has a better effectiveness at detecting low-rate targeted injection attacks as illustrated in 8. In addition, frequency based approach is also a window based approach, thus will be able to detect anomalies about 128 milliseconds earlier than subsequence mining. However, the F-score of subsequence mining is 15.08% better than frequency based approaches and 296% better than dictionary based approaches. We consider this a minor increase in computation time a valid trade-off for higher accuracy.

G. EFFECT OF MAXIMUM LENGTH OF SUBSEQUENCE

We also evaluate the impact of the maximum sequence length on the performance of the sequence mining approach. Figure 9 provides the overview of the f-score while detecting anomalies for targeted replay injections on the primary and EV CAN bus. F-score represents the harmonic mean between precision and recall. The results show that the f-score increases as the maximum subsequence length is increased. The f-score increases from 91.05 for $n = 2$ to detect denial



(a) Primary CAN

(b) EV CAN

FIGURE 10. Total number of sequences generated from various CAN data for different values of n.

of service attack to 95.57 for $n = 8$. There is no impact on the detection of true positives or precision as the value of n is increased. The recall value increases as the value of n is increased. As value of n is increased the number of sequences also increases, thus creating more specific combinations of valid message ids. This enables the knowledge dictionary to accurately identify anomalous patterns. The recall plateaus after $n = 5$, thus increasing the maximum length of the subsequence beyond $n = 8$ is counter productive as it increases the time taken to generate the subsequence rules and their support values.

Figure 10 presents the total number of unique sequences generated for various values of n. As the value of n increases, the number of sequences generated also increases. The increase in the sequences are cumulative, with n=8 containing any sequence of lengths $n < 8$, if that rule is not encompassed by a super-pattern with the same support. Thus on the primary CAN, the number of subsequences increases from 221 for $n = 2$ to 3295 for $n = 8$. On the EV CAN, the total number of subsequences is about 1842 for $n = 8$. The results show that as the value of n increases the ability of the sequence mining approach to identify much more complex rules that describe the normal behavior of the vehicle also increases. Thus the capability of the sequence mining approach to detect unusual patterns with frequencies that do not match their original behavior.

Table 3 presents the total time taken to generate all the sequences and their support values for various values of n and the average time taken to evaluate a test sequence. The training and testing time increases exponentially as the maximum subsequence length increases. As the maximum subsequence length increase, the size of the suffix tree increases leading to longer time taken to parse the tree and calculate the support values. The training time increases from 8 seconds to about 38 seconds for the training data. The time taken

TABLE 3. Training and Testing time to generate sequences and detect anomalies for various values of n.

n	Training Time (sec)	Detection Time (sec)
2	8.04	0.151
3	10.14	0.178
4	13.51	0.191
5	18.31	0.236
6	23.15	0.353
7	29.44	0.491
8	38.52	0.686

to detect anomalies from the test data also increases with the increase in maximum subsequence length. While we can detect anomalies in data in less than a second. As the results show, the increase in value of n increases the effectiveness of the anomaly detection on the CAN bus data, however, the cost to detect those anomalies also increase as we increase the value of n. The results show that the time taken to detect subsequences is lower on the EV CAN compared to primary CAN. Thus, an increased n value is recommended for CAN with small number of unique message IDs and low frequency data while a CAN bus with higher frequency and large number of message ids cannot be processed in real-time.

H. DISCUSSION AND LIMITATIONS

The training model built during the training phase is used to identify intrusions in CAN bus in real-time. This training model is valid as long as the configuration of the vehicle remains the same and needs to be retrained if the configuration of the vehicle is changed. Some changes to the configuration include adding a new ECU to the vehicle, damage to a ECU or removal of an ECU. The model will have to be retrained if any of the aforementioned changes are noticed in the vehicle.

One major constraint on the subsequence mining is the detection time. In order to detect intrusions in real-time, the detection time should be lower than the size of the tumbling window. However, our results indicate that the performance of the approach increases with the maximum subsequence size. This determination should be made on a case by case basis as this parameter is dependent on the number of total messages and unique message ids in the CAN bus in a given time period. A determined attacker with access to the vehicle as well as knowledge of the intrusion detection approach can develop a brute force approach to identify subsequences that allow the attacker to launch attacks that cannot be detected. However, finding those sequences is computationally expensive, but not impossible. One possible solution would be to use multiple training models using random combinations of multiple subsequence lengths to improve security of the approach.

VII. CONCLUSION

We proposed pattern-frequency based subsequence mining approach to detect replay injections in CAN bus messages. The sequence mining technique was able to achieve an f-score between 99.07% and 99.7% for sequence replay injections. We compared the performance of the sequence mining approach against dictionary based approach, multivariate Markov chain process, and frequency based approaches. The sequence mining approach outperforms the dictionary based approach, Markov chain process, and frequency based approaches for injection length of 5 and attack frequency of 5 with a f-score of 99.42%, 73.05%, 84.43%, and 98.66% respectively. We also compare the performance against four types of replay attacks: dominant id replay, random replay, sequence replay, and targeted reply attacks. The sequence mining approach performs as well or better than the baseline techniques. We compare the minimum time window required to detect the injections in the CAN bus data, the minimum window length should be greater than the inter-message frequency. The time taken to evaluate one second of data for injection is lower than the time taken to gather the data for subsequence length, and can be evaluated in real-time. Thus, sequence mining approach can detect sophisticated injection attacks that are highly targeted, has small injection length and lower attack frequency, providing real-time intrusion detection for the vehicle.

The computation cost to build subsequences is higher compared to computing frequencies and dictionary. In future, we would like to reduce the computation cost to generate subsequences. Our results indicate that 45 minutes of training data is sufficient for reliable intrusion detection. We would like to study the impact of the training data size on detection performance. We also would like to identify various subsequences that are prevalent during various states of the vehicle (changing gears, idle, accelerating, turning etc.). These would help us improve intrusion detection by building a subsequence dictionary to tailor sequences to a specific state of the vehicle.

ACKNOWLEDGMENT

The authors would like to thank the members of the cyber-physcial systems lab at the University of Louisiana at Lafayette for their help in collecting the data for this manuscript. The authors also thank the reviewers for their feedback.

REFERENCES

[1] P. Carsten, T. R. Andel, M. Yampolskiy, and J. T. McDonald, "In-vehicle networks: Attacks, vulnerabilities, and proposed solutions," in *Proc. 10th Annu. Cyber Inf. Secur. Res. Conf. (CISR)*, 2015, p. 1.

[2] S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, S. Savage, K. Koscher, A. Czeskis, F. Roesner, and T. Kohno, "Comprehensive experimental analyses of automotive attack surfaces," in *Proc. USENIX Secur. Symp.*, San Francisco, CA, USA, vol. 4, 2011, pp. 447–462.

[3] I. Studnia, V. Nicomette, E. Alata, Y. Deswarte, M. Kaaniche, and Y. Laarouchi, "Survey on security threats and protection mechanisms in embedded automotive networks," in *Proc. 43rd Annu. IEEE/IFIP Conf. Dependable Syst. Netw. Workshop (DSN-W)*, Jun. 2013, pp. 1–12.

[4] H. Lee, S. H. Jeong, and H. K. Kim, "OTIDS: A novel intrusion detection system for in-vehicle network by using remote frame," in *Proc. 15th Annu. Conf. Privacy, Secur. Trust (PST)*, Aug. 2017, pp. 57–5709.

[5] F. Lambert. (2019). *Hackers Crack Tesla Model 3 in Competition, Tesla Gives Them the Car*. [Online]. Available: https://electrek.co/2019/03/23/tesla-model-3-hacker-competition-crack/

[6] M. R. Moore, R. A. Bridges, F. L. Combs, M. S. Starr, and S. J. Prowell, "Modeling inter-signal arrival times for accurate detection of can bus signal injection attacks: A data-driven approach to in-vehicle intrusion detection," in *Proc. 12th Annu. Conf. Cyber Inf. Secur. Res.*, 2017, p. 11.

[7] A. Taylor, N. Japkowicz, and S. Leblanc, "Frequency-based anomaly detection for the automotive CAN bus," in *Proc. World Congr. Ind. Control Syst. Secur. (WCICSS)*, Dec. 2015, pp. 45–49.

[8] M. Marchetti and D. Stabili, "Anomaly detection of CAN bus messages through analysis of ID sequences," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2017, pp. 1577–1583.

[9] A. Taylor, S. Leblanc, and N. Japkowicz, "Probing the limits of anomaly detectors for automobiles with a cyberattack framework," *IEEE Intell. Syst.*, vol. 33, no. 2, pp. 54–62, Mar. 2018.

[10] W. Wu, Y. Huang, R. Kurachi, G. Zeng, G. Xie, R. Li, and K. Li, "Sliding window optimized information entropy analysis method for intrusion detection on in-vehicle networks," *IEEE Access*, vol. 6, pp. 45233–45245, 2018.

[11] T. Kuwahara, Y. Baba, H. Kashima, T. Kishikawa, J. Tsurumi, T. Haga, Y. Ujiie, T. Sasaki, and H. Matsushima, "Supervised and unsupervised intrusion detection based on CAN message frequencies for in-vehicle network," *J. Inf. Process.*, vol. 26, no. 0, pp. 306–313, 2018.

[12] A. Kuzmanovic and E. W. Knightly, "Low-rate TCP-targeted denial of service attacks: The shrew vs. the mice and elephants," in *Proc. Conf. Appl., Technol., Archit., Protocols Comput. Commun.*, 2003, pp. 75–86.

[13] H. Sun, J. C. S. Lui, and D. K. Y. Yau, "Defending against low-rate TCP attacks: Dynamic detection and protection," in *Proc. 12th IEEE Int. Conf. Netw. Protocols (ICNP)*, Oct. 2004, pp. 196–205.

[14] T. Hoppe, S. Kiltz, A. Lang, and J. Dittmann, "Exemplary automotive attack scenarios: Trojan horses for electronic throttle control system (ETC) and replay attacks on the power window system," *VDI BERICHTE*, vol. 23, p. 165, Nov. 2016.

[15] T. Hoppe, S. Kiltz, and J. Dittmann, "Security threats to automotive CAN networks—Practical examples and selected short-term countermeasures," *Rel. Eng. Syst. Saf.*, vol. 96, no. 1, pp. 11–25, 2011.

[16] A. Greenberg. (2015). *Hackers Remotely Kill a Jeep on the Highway-With Me in it*. [Online]. Available: https://www.wired.com/2015/07/hackers-remotely-kill-jeep-highway/

[17] C. Miller and C. Valasek. (2016). *Can Message Injection: OG Dynamite Edition*. [Online]. Available: http://illmatics.com/canmessageinjection.pdf

[18] K. Koscher, A. Czeskis, F. Roesner, S. Patel, T. Kohno, S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, and S. Savage, "Experimental security analysis of a modern automobile," in *Proc. IEEE Symp. Secur. Privacy*, May 2010, pp. 447–462.

[19] J. Petit and S. E. Shladover, "Potential cyberattacks on automated vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 2, pp. 546–556, Apr. 2015.

[20] P. Sun, S. Chawla, and B. Arunasalam, "Mining for outliers in sequential databases," in *Proc. SIAM Int. Conf. Data Mining*. Philadelphia, PA, USA: SIAM, Apr. 2006, pp. 94–105.
[21] S. Chakrabarti, S. Sarawagi, and B. Dom, "Mining surprising patterns using temporal description length," in *Proc. VLDB*, vol. 98, 1998, pp. 606–617.
[22] D. Y. Pavlov and D. M. Pennock, "A maximum entropy approach to collaborative filtering in dynamic, sparse, high-dimensional domains," in *Proc. Adv. Neural Inf. Process. Syst.*, 2003, pp. 1465–1472.
[23] O. Y. Al-Jarrah, C. Maple, M. Dianati, D. Oxtoby, and A. Mouzakitis, "Intrusion detection systems for intra-vehicle networks: A review," *IEEE Access*, vol. 7, pp. 21266–21289, 2019.
[24] H. Ji, Y. Wang, H. Qin, Y. Wang, and H. Li, "Comparative performance evaluation of intrusion detection methods for in-vehicle networks," *IEEE Access*, vol. 6, pp. 37523–37532, 2018.
[25] M. Gmiden, M. H. Gmiden, and H. Trabelsi, "An intrusion detection method for securing in-vehicle CAN bus," in *Proc. 17th Int. Conf. Sci. Techn. Autom. Control Comput. Eng. (STA)*, Dec. 2016, pp. 176–180.
[26] A. Taylor, "Anomaly-based detection of malicious activity in in-vehicle networks," Ph.D. dissertation, Univ. Ottawa, Ottawa, ON, Canada, 2017.
[27] Y. Laarouchi, M. Kaâniche, V. Nicomette, I. Studnia, and E. Alata, "A language-based intrusion detection approach for automotive embedded networks," *Int. J. Embedded Syst.*, vol. 10, no. 1, p. 1, 2018.
[28] S. N. Narayanan, S. Mittal, and A. Joshi, "OBD_SecureAlert: An anomaly detection system for vehicles," in *Proc. IEEE Int. Conf. Smart Comput. (SMARTCOMP)*, May 2016, pp. 1–6.
[29] A. Taylor, S. Leblanc, and N. Japkowicz, "Anomaly detection in automobile control network data with long short-term memory networks," in *Proc. IEEE Int. Conf. Data Sci. Adv. Anal. (DSAA)*, Oct. 2016, pp. 130–139.
[30] L. Zhang, L. Shi, N. Kaja, and D. Ma, "A two-stage deep learning approach for can intrusion detection," in *Proc. Ground Vehicle Syst. Eng. Technol. Symp.(GVSETS)*, 2018, pp. 1–11.
[31] S. Garg, K. Kaur, N. Kumar, S. Batra, and M. S. Obaidat, "HyClass: Hybrid classification model for anomaly detection in cloud environment," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2018, pp. 1–7.
[32] S. Garg, K. Kaur, N. Kumar, G. Kaddoum, A. Y. Zomaya, and R. Ranjan, "A hybrid deep learning-based model for anomaly detection in cloud datacenter networks," *IEEE Trans. Netw. Service Manage.*, vol. 16, no. 3, pp. 924–935, Sep. 2019.
[33] S.-H. Chen and C.-H. R. Lin, "Evaluation of DoS attacks on vehicle CAN bus system," in *Proc. Int. Conf. Intell. Inf. Hiding Multimedia Signal Process.* Sendai, Japan: Springer, 2018, pp. 308–314.
[34] P. Fournier-Viger, A. Gomariz, M. Šebek, and M. Hlosta, "VGEN: Fast vertical mining of sequential generator patterns," in *Proc. Int. Conf. Data Warehousing Knowl. Discovery*. Munich, Germany: Springer, 2014, pp. 476–488.
[35] J. Wang and J. Han, "BIDE: Efficient mining of frequent closed sequences," in *Proc. 20th Int. Conf. Data Eng.*, Apr. 2004, pp. 79–90.
[36] W.-K. Ching, M. K. Ng, and E. S. Fung, "Higher-order multivariate Markov chains and their applications," *Linear Algebra Appl.*, vol. 428, nos. 2–3, pp. 492–507, Jan. 2008.
[37] M. Elhamahmy, H. N. Elmahdy, and I. A. Saroit, "A new approach for evaluating intrusion detection system," *CiiT Int. J. Artif. Intell. Syst. Mach. Learn.*, vol. 2, no. 11, pp. 290–298, 2010.

SATYA KATRAGADDA received the Ph.D. degree in computer science from the University of Louisiana at Lafayette. He is currently a Research Scientist with the Informatics Research Institute, University of Louisiana at Lafayette. He also develops techniques to ingest, process, model, and visualize data streams. His area of research is the area of processing real-time data streams.

PAUL J. DARBY, III (Member, IEEE) received the B.S. degree in electrical engineering and the M.S. degree in telecommunications from the University of Louisiana at Lafayette and the Ph.D. degree in computer engineering from the Center for Advanced Computer Studies, University of Louisiana at Lafayette. His Ph.D. advisor was Dr. Nian-Feng Tzeng, IEEE Fellow. He was actively engaged in both the aerospace and telecommunications industries for more than 20 years. In telecommunications, he served as an Engineer Advanced for the State of Louisiana at the Department of Public Safety and the Office of Telecommunications Management, including LaNet. He has been a registered Professional Engineer, since 1989. He currently serves as an Assistant Professor of electrical and computer engineering at the Department of Electrical and Computer Engineering, University of Louisiana at Lafayette. His research interests include mobile wireless grid computing, satellite technology, and cyber-physical programming and security. He was a recipient of the NASA's Silver Snoopy Award.

ANDREW ROCHE received the M.S. degree in electrical engineering from the University of Louisiana at Lafayette. He currently serves as an Instructor of electrical and computer engineering at the Department of Electrical and Computer Engineering, University of Louisiana at Lafayette.

RAJU GOTTUMUKKALA received the Ph.D. degree in computer science from Louisiana Tech University, LA, USA. He is currently the Director of the Research for the Informatics Research Institute, University of Louisiana at Lafayette. He is also the Site Director of NSF CVDI—an NSF Industry/University Cooperative Research Center in the area of big data. He has led various efforts in the area of big data platforms, system resilience, modeling and verification of distributed systems, software-defined networks, visual analytics, and evolutionary networks. His research interest is in the broader area of cyber physical systems, specifically addressing real-world informatics and integrated systems modeling issues.